

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Vasilean Vitalie
Group FAF-241

Report

Laboratory Work No. 2
Operating Systems
Topic: Meet the OS You Will Build (xv6)

Checked by:
univ. assist. / lect. Patricia Reițman
Department of Software Engineering and Automation
FCIM, UTM

Chisinau – 2026

Contents

1 INTRODUCTION AND GOAL **3**

1.1 Introduction 3

1.2 Lab Objective 3

2 PART 1: SETUP AND BUILD ENVIRONMENT **4**

2.1 Toolchain Verification and Compilation 4

2.2 Analysis and Observations 4

3 PART 2: USING XV6 AS THE UNIX IT IS **5**

3.1 Command Execution and Shell Interaction 5

3.2 Answers to Required Observation Questions 5

4 PART 3: READING THE SOURCE CODE **7**

4.1 Inspection of Userland Source Code (`user/cat.c`) 7

4.2 System Call Definitions and Table Mappings 7

4.3 Answers to Required Observation Questions 8

5 PART 4: IMPLEMENTING THE SLEEP UTILITY **10**

5.1 Implementation of `user/sleep.c` 10

5.2 Makefile Configuration and Build Integration 10

5.3 Verification and Execution Output 11

6 CONCLUSIONS **12**

1 INTRODUCTION AND GOAL

1.1 Introduction

Modern monolithic production kernels such as Linux encompass millions of lines of code, complicating direct pedagogical study of kernel subsystems. In contrast, xv6 is a clean, modern re-implementation of Dennis Ritchie's and Ken Thompson's classic Unix Version 6 (V6), ported to the 64-bit RISC-V architecture. xv6 illustrates foundational concepts—virtual memory paging, process context switches, hardware interrupts, system call traps, and file system disk blocks—in a concise, readable codebase.

1.2 Lab Objective

The primary objective of this laboratory work is to configure the xv6 development toolchain, boot the operating system under hardware emulation, interact with its userland environment, study core system call interfaces at the source code level, and implement a custom userland utility. The specific scope comprises:

- establishing the cross-compilation toolchain (`riscv64-linux-gnu-gcc`) and booting xv6 in the QEMU emulator;
- interacting with standard xv6 utilities (`ls`, `cat`, `echo`, `grep`, `wc`) and evaluating command pipelines;
- answering core architectural observation questions regarding process isolation, stream composition, and userland-versus-kernel separation;
- analyzing utility source code implementations (`user/cat.c`) and tracing user-to-kernel transitions via system call tables (`user/user.h`, `kernel/syscall.h`, `kernel/sysfile.c`);
- developing, integrating, and verifying the `sleep` utility in `user/sleep.c`.

2 PART 1: SETUP AND BUILD ENVIRONMENT

2.1 Toolchain Verification and Compilation

The RISC-V cross-compiler and QEMU system emulator were verified under Ubuntu WSL2. The xv6 repository was cloned, and the operating system was compiled using the native `Makefile`:

```
vitalie@Lenovo:~$ riscv64-linux-gnu-gcc --version | head -n 1
riscv64-linux-gnu-gcc (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0

vitalie@Lenovo:~$ qemu-system-riscv64 --version | head -n 1
QEMU emulator version 8.2.2 (Debian 1:8.2.2+ds-0ubuntu1.2)

vitalie@Lenovo:~$ cd ~/xv6-riscv && make qemu
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -gdwarf-2 -march=rv64gc -
    MD -mcmodel=medany -ffreestanding -c -o kernel/entry.o kernel/entry.S
...
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -
    global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device
    virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$
```

Figure 2.1: Figure 2.1 – xv6 compilation and initial boot banner under QEMU RISC-V emulator

2.2 Analysis and Observations

a) **Cross-compilation necessity:** Because the host machine runs an x86_64 processor while xv6 targets the RISC-V (rv64gc) architecture, a cross-compiler (`riscv64-linux-gnu-gcc`) is mandatory to generate valid machine instructions.

b) **SMP and Multi-hart startup:** During boot, xv6 initializes hardware thread 0 (hart 0), which configures core memory mappings, and subsequently brings up secondary cores (`hart 1 starting`, `hart 2 starting`). Once kernel subsystems are online, the kernel spawns the userland `init` process, which in turn launches the shell (`sh`).

3 PART 2: USING XV6 AS THE UNIX IT IS

3.1 Command Execution and Shell Interaction

Standard utilities and anonymous pipes were executed at the xv6 interactive shell prompt:

```
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 36728
echo           2 4 35592
forktest       2 5 18136
grep           2 6 39792
init           2 7 35832
kill           2 8 35584
ln             2 9 35432
ls             2 10 38048
mkdir          2 11 35640
rm             2 12 35632
sh             2 13 56840
stressfs       2 14 36608
usertests      2 15 192800
grind          2 16 50744
wc             2 17 37192
zombie         2 18 35056

$ echo hello xv6
hello xv6

$ wc README
94 656 2441 README

$ ls | grep c
cat
echo
forktest
stressfs
usertests
wc
```

Figure 3.1: Figure 3.1 – Directory listing, utility execution, and pipe chaining in xv6

3.2 Answers to Required Observation Questions

a) **Three programs xv6 ships with:** As evidenced by the directory listing produced by `ls`, xv6 ships out-of-the-box with fundamental Unix tools including:

- `ls` (file listing utility displaying type, inode number, and size);
- `cat` (file concatenation and output stream reader);
- `echo` (argument echo utility).

Additional tools include `grep`, `wc`, `mkdir`, `rm`, and `kill`.

b) **OS features required for pipes (`ls | grep c`):** For a pipeline command such as `ls | grep c` to execute correctly, the operating system must provide two foundational capabilities:

- **Isolated Concurrent Processes:** The kernel must support multitasking via process abstraction and address space isolation (`fork()`). The shell spawns two child processes that execute concurrently rather than sequentially.
- **Inter-Process Communication (IPC) via Pipes and Descriptor Redirection:** The kernel must supply an IPC buffer mechanism (`pipe()`) allocated in kernel memory, along with descriptor table remapping primitives (`dup()` and `close()`). The write end of the pipe replaces file descriptor 1 (`stdout`) of the producer process (`ls`), and the read end replaces file descriptor 0 (`stdin`) of the consumer process (`grep`).

c) **Comparison of xv6 Shell and Modern Linux Shell:**

- **Shared Philosophy:** Both shells adhere to the classic Unix design: modular programs composed through byte streams, process creation via `fork` and `exec`, and standard I/O redirection (<, >, |).
- **Differences in Complexity:** The xv6 shell (`user/sh.c`) is intentionally compact (under 500 lines of code) to demonstrate core OS primitives. Unlike Linux shells (such as Bash or Zsh), the xv6 shell lacks environment variable expansion (e.g., `$PATH`, `$HOME`), command line history, tab completion, job control (`Ctrl+Z`, background jobs `&`), aliases, globbing wildcards (`*.c`), and complex programming constructs (loops, conditionals, functions).

4 PART 3: READING THE SOURCE CODE

4.1 Inspection of Userland Source Code (`user/cat.c`)

The source code of `user/cat.c` was examined to observe how unprivileged user programs request kernel services:

```
// user/cat.c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

char buf[512];

void
cat(int fd)
{
  int n;

  while((n = read(fd, buf, sizeof(buf))) > 0) {
    if (write(1, buf, n) != n) {
      fprintf(2, "cat: write error\n");
      exit(1);
    }
  }
  if(n < 0){
    fprintf(2, "cat: read error\n");
    exit(1);
  }
}

int
main(int argc, char *argv[])
{
  int fd, i;

  if(argc <= 1){
    cat(0);
    exit(0);
  }

  for(i = 1; i < argc; i++){
    if((fd = open(argv[i], 0)) < 0){
      fprintf(2, "cat: cannot open %s\n", argv[i]);
      exit(1);
    }
    cat(fd);
    close(fd);
  }
  exit(0);
}
```

Figure 4.1: Figure 4.1 – Implementation of user/cat.c in xv6

4.2 System Call Definitions and Table Mappings

System call declarations and dispatcher numbers were inspected in `user/user.h` and `kernel/syscall.h`:

```c
// user/user.h (system call prototypes available to userland)
int read(int, void*, int);
int write(int, const void*, int);
int open(const char*, int);
int close(int);
int exit(int) __attribute__((noreturn));
```

Figure 4.2: Figure 4.2 – System call prototypes in user/user.h

```c
// kernel/syscall.h (system call identifiers)
#define SYS_fork    1
#define SYS_exit    2
#define SYS_wait    3
#define SYS_pipe    4
#define SYS_read    5
#define SYS_kill    6
#define SYS_exec    7
#define SYS_fstat   8
#define SYS_chdir   9
#define SYS_dup    10
#define SYS_getpid 11
#define SYS_sbrk   12
#define SYS_pause  13
#define SYS_uptime 14
#define SYS_open   15
#define SYS_write  16
#define SYS_mknod  17
#define SYS_unlink 18
#define SYS_link   19
#define SYS_mkdir  20
#define SYS_close  21
```

Figure 4.3: Figure 4.3 – Numeric system call vector mapping in kernel/syscall.h

4.3 Answers to Required Observation Questions

a) **System calls utilized by `cat.c`:** Inspection of `cat.c` reveals the explicit invocation of five system calls:

- `open(path, mode)` — opens input files for reading;
- `read(fd, buf, size)` — reads data chunks up to 512 bytes;
- `write(1, buf, n)` — outputs read buffers to standard output (or descriptor 2 for errors);
- `close(fd)` — closes the file descriptor upon EOF;
- `exit(status)` — terminates execution cleanly.

b) **Kernel location of `sys_read`:** The kernel implementation of `sys_read` is located in:

- **File:** `kernel/sysfile.c`
- **Exact Line:** Line 69:

```
uint64
sys_read(void)
{
  struct file *f;
  int n;
  uint64 p;

  argfd(0, 0, &f);
  argaddr(1, &p);
  argint(2, &n);
  return fileread(f, p, n);
}
```

This function unmarshals user parameters from trapframe registers via `argfd`, `argaddr`, and `argint`, validates pointer safety, and delegates the read operation to `fileread()`.

c) **Structural and Privilege Separation Between `kernel/` and `user/`:**

- **Hardware Privilege Levels:** The code in `kernel/` executes in RISC-V **Supervisor Mode**, granting it access to privileged control registers (CSRs such as `sstatus`, `satp`, `stvec`), page table manipulation, device MMIO, and interrupt control. In contrast, code in `user/` executes in RISC-V **User Mode** and is restricted to its own virtual address space.
- **Transition Boundary:** User code cannot invoke kernel functions directly. It must execute an assembly stub generated in `user/usys.S`, which loads the syscall constant into register `a7` and triggers an `ecall` instruction. The hardware raises an environment call trap, transitioning the CPU to supervisor mode where `kernel/syscall.c` dispatches the request.

5 PART 4: IMPLEMENTING THE SLEEP UTILITY

5.1 Implementation of `user/sleep.c`

In accordance with the lab specifications, a custom userland utility `sleep` was implemented. The utility accepts a single argument specifying the duration in clock ticks, pauses the calling process, and terminates:

```
// user/sleep.c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2) {
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1])); // xv6 sleep/pause system call
  exit(0);
}
```

Figure 5.1: Figure 5.1 – Source code of user/sleep.c

5.2 Makefile Configuration and Build Integration

To enable compilation and include `sleep` within the generated disk image (`fs.img`), `$U/_sleep\` was registered in the `UPROGS` list in the xv6 `Makefile`:

```
UPROGS=\
    $U/_cat\
    $U/_echo\
    $U/_forktest\
    $U/_grep\
    $U/_init\
    $U/_kill\
    $U/_ln\
    $U/_ls\
    $U/_mkdir\
    $U/_rm\
    $U/_sh\
    $U/_stressfs\
    $U/_usertests\
    $U/_grind\
    $U/_wc\
    $U/_zombie\
    $U/_sleep\
```

Figure 5.2: Figure 5.2 – Integration of user/sleep into the xv6 Makefile

5.3 Verification and Execution Output

The operating system was recompiled and launched in QEMU to verify utility behavior under correct invocation and erroneous parameter input:

```
$ make qemu
...
xv6 kernel is booting
init: starting sh

$ sleep 20
# Process pauses for 20 timer ticks (approx. 2 seconds) and returns prompt

$ sleep
usage: sleep <ticks>
$
```

Figure 5.3: Figure 5.3 – Interactive verification of sleep utility inside xv6

6 CONCLUSIONS

In this laboratory work, the architecture of the xv6-riscv educational operating system was investigated through environment deployment, execution testing, source code inspection, and utility implementation.

Booting xv6 in QEMU confirmed how an operating system initializes multi-core processor contexts and establishes userland execution with a clean terminal prompt. Interacting with basic commands demonstrated that fundamental Unix abstractions—including standard streams, descriptor redirection, anonymous pipes, and hierarchical namespaces—operate identically in xv6 and production Linux despite vast differences in code size.

Inspecting `user/cat.c`, `user/user.h`, and `kernel/syscall.h` clarified the complete lifecycle of a system call trap from userland execution down to kernel service routines, specifically pinpointing `sys_read` in `kernel/sysfile.c`. Finally, the implementation and integration of the `sleep` utility in `user/sleep.c` validated the workflow of extending the xv6 userland with custom software utilities.